Aim: Introduction to Truffle, Establish local blockchain using Truffle and deployment of Smart Contract

Tasks Performed:

- Installed Ganache to run a local blockchain.
- Compiled Migrations.sol and SimpleStorage.sol with `npx truffle compile`.
- Migrated (deployed) both contracts to the Ganache "development" network, which made 2 deployments.
- Ran `interact.js` to send the transaction `set(100)`, then called `get()`, which returned 100.
- Checked the transactions and blocks (up to block 10) in the Ganache interface.

Theory:

Local Blockchains – Developing DApps directly on the Ethereum mainnet or public testnets brings latency, rate limits and gas costs. A local private blockchain simulates the EVM on a personal workstation with instant block mining, pre-funded accounts and deterministic state, so developers can iterate quickly without risking real assets.

Truffle Suite – A development environment, testing framework and asset pipeline for EVM blockchains. Its core capabilities are:

- **Compilation –** converts Solidity (.sol) into EVM bytecode and ABI.
- **Migrations –** scriptable deployment that tracks earlier deployments on-chain.
- **Network management –** local, testnet and mainnet settings kept in truffle-config.js.
- **Console and scripts –** truffle console and truffle exec run Node.js code with Web3.

A Truffle project contains contracts/ (Solidity sources and Migrations.sol), migrations/ (numbered deployment scripts), test/ (automated tests), build/contracts/ (compiled JSON artifacts) and truffle-config.js (network, gas and compiler settings).

Ganache – The local EVM simulator of the Truffle Suite. It runs a private Ethereum node exposed over JSON-RPC (typically http://127.0.0.1:7545 or 8545). It generates 10 deterministic accounts from a mnemonic, each funded with 100 test ETH, mines blocks instantly (automine), exposes private keys for import into wallets such as MetaMask, and allows near-zero gas prices so testing does not drain balances.

Bytecode and ABI – Bytecode is the low-level code executed by the EVM when a contract is created or called. The Application Binary Interface (ABI) is a JSON description of the contract's public functions, arguments, return values and events, which lets scripts and libraries such as Web3.js or Ethers.js encode calls and decode results.

Truffle Migrations – A Migrations.sol contract stores last_completed_migration. When truffle migrate runs, only scripts numbered above that value are executed, which prevents duplicate

deployments. The deployer object handles deployment order, library linking and constructor arguments.

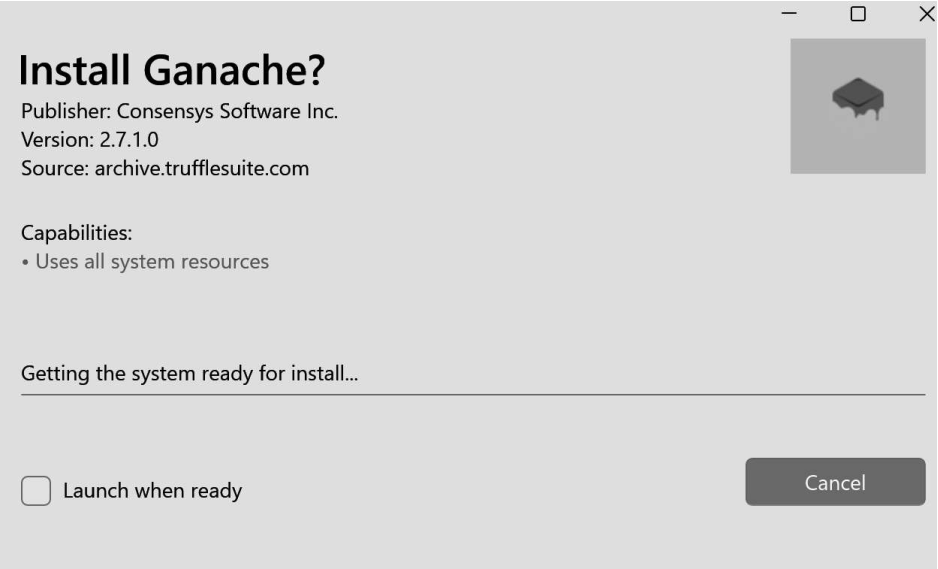
Calls vs Transactions –

- **Calls** (view/pure functions such as get()) are read-only, run locally on the node, change no state and cost no gas.
- **Transactions** (state-changing functions such as set(uint x)) must be signed by a sender, are mined into a new block, receive a transaction hash and consume gas.

Code & Output:

Step 1 — Install Ganache

Ganache 2.7.1.0 (Consensys) is installed to provide the local blockchain.



Step 2 — Compile the smart contracts

npx truffle compile

Migrations.sol and SimpleStorage.sol compile successfully with solc 0.8.20 and the artifacts are written to build\contracts.

```
PS C:\Users\heram\Desktop\Exp_7_blockchain> npx truffle compile
>>

Compiling your contracts...
===========================
> Compiling .\contracts\Migrations.sol
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\heram\Desktop\Exp_7_blockchain\build\contracts
> Compiled successfully using:
   - solc: 0.8.20+commit.a1b79de6.Emscripten.clang
```

Step 3 — Migrate (deploy) the contracts to Ganache

npx truffle migrate

The “development” network (network id 5777) is used. 1_initial_migration.js deploys Migrations, and 2_deploy_contracts.js then deploys SimpleStorage at 0xe25074697d309198a22dB4E8256893439f56C4FE.

```
Compiling your contracts...
===========================
> Compiling .\contracts\Migrations.sol
> Compiling .\contracts\SimpleStorage.sol
> Artifacts written to C:\Users\heram\Desktop\Exp_7_blockchain\build\contracts
> Compiled successfully using:
   - solc: 0.8.20+commit.a1b79de6.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)


1_initial_migration.js
======================

   Replacing 'Migrations'
   ----------------------
   > transaction hash:    0x7f8bd17c70dd2bded2987e5042b4dc55af30c5bf71d60320312e8eeb2467a335
   > Blocks: 0            Seconds: 0
   > contract address:    0x5c6ec88fF9A3A9677F1e01B4f894F87Fb84C4ff6
   > block number:        6
   > block timestamp:     1791389013
   > account:             0xc8637eBF4F8e148B2CAe4Bf5D0C844D60836c2C7
   > balance:             99.99764469459288602
   > gas used:            250154 (0x3d12a)
   > gas price:           2.958288746 gwei
   > value sent:          0 ETH
```

```
   > gas price:           2.958288746 gwei
   > value sent:          0 ETH
   > total cost:          0.000740027762966884 ETH

   > Saving migration to chain.
   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000740027762966884 ETH


2_deploy_contracts.js
=====================

   Replacing 'SimpleStorage'
   > Blocks: 0            Seconds: 0
   > contract address:    0xe25074697d309198a22dB4E8256893439f56C4fE
   > block number:        8
   > block timestamp:     1791389013
   > account:             0xc8637eBF4F8e148B2CAe4Bf5D0C844D60836c2C7
   > balance:             99.997152528072755338
   > gas used:            125653 (0x1ead5)
   > gas price:           2.855300109 gwei
   > value sent:          0 ETH
   > total cost:          0.000358777024596177 ETH

   > Saving migration to chain.
   > Saving artifacts
   -------------------------------------
   > Total cost:     0.000358777024596177 ETH
```

Step 4 — Deployment summary

Two deployments were made at a final cost of about 0.0011 ETH.

```
Summary
=======
> Total deployments:   2
> Final cost:          0.001098804787563061 ETH


PS C:\Users\heram\Desktop\Exp_7_blockchain>
```

Step 5 — Interact with the contract using a script

npx truffle exec scripts/interact.js

The script connects to the deployed SimpleStorage, sends the transaction set(100) (block 10, gas used 43702) and then calls get(), which returns the stored value 100. The uWS warning is harmless because Ganache falls back to a Node.js implementation.

```
PS C:\Users\heram\Desktop\Exp_7_blockchain> npx truffle exec scripts/interact.js
>>
This version of µWS is not compatible with your Node.js build:

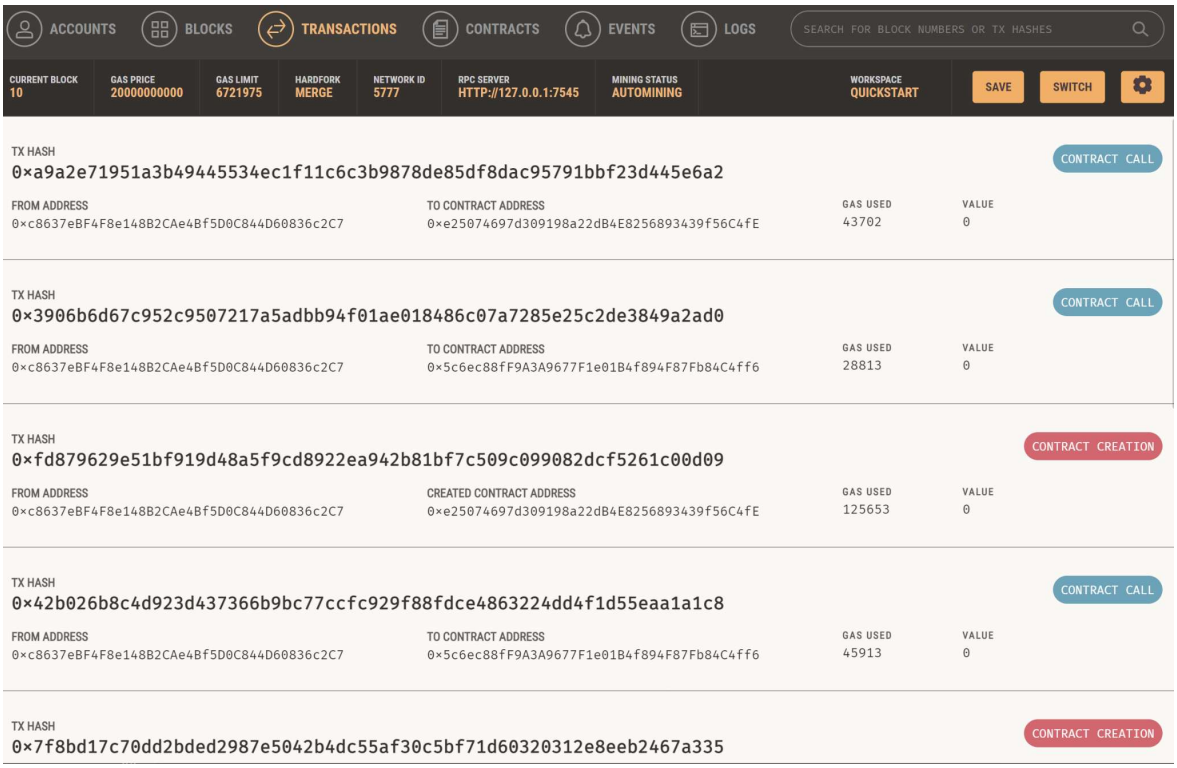
Error: Cannot find module '../binaries/uws_win32_x64_137.node'
Require stack:
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\ganache\node_modules\@trufflesuite\uws
-js-unofficial\src\uws.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\ganache\dist\node\core.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\truffle\build\exec.bundled.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\original-require\index.js
- C:\Users\heram\Desktop\Exp_7_blockchain\node_modules\truffle\build\cli.bundled.js
Falling back to a NodeJS implementation; performance may be degraded.


Using network 'development'.

------------------------------------------------
Connected to deployed SimpleStorage at: 0xe25074697d309198a22dB4E8256893439f56C4fE
Executing transaction: set(100)...
Transaction Hash: 0xa9a2e71951a3b49445534ec1f11c6c3b9878de85df8dac95791bbf23d445e6a2
Block Number: 10
Gas Used: 43702
Calling get()...
Current Stored Value: 100
------------------------------------------------
PS C:\Users\heram\Desktop\Exp_7_blockchain>
```

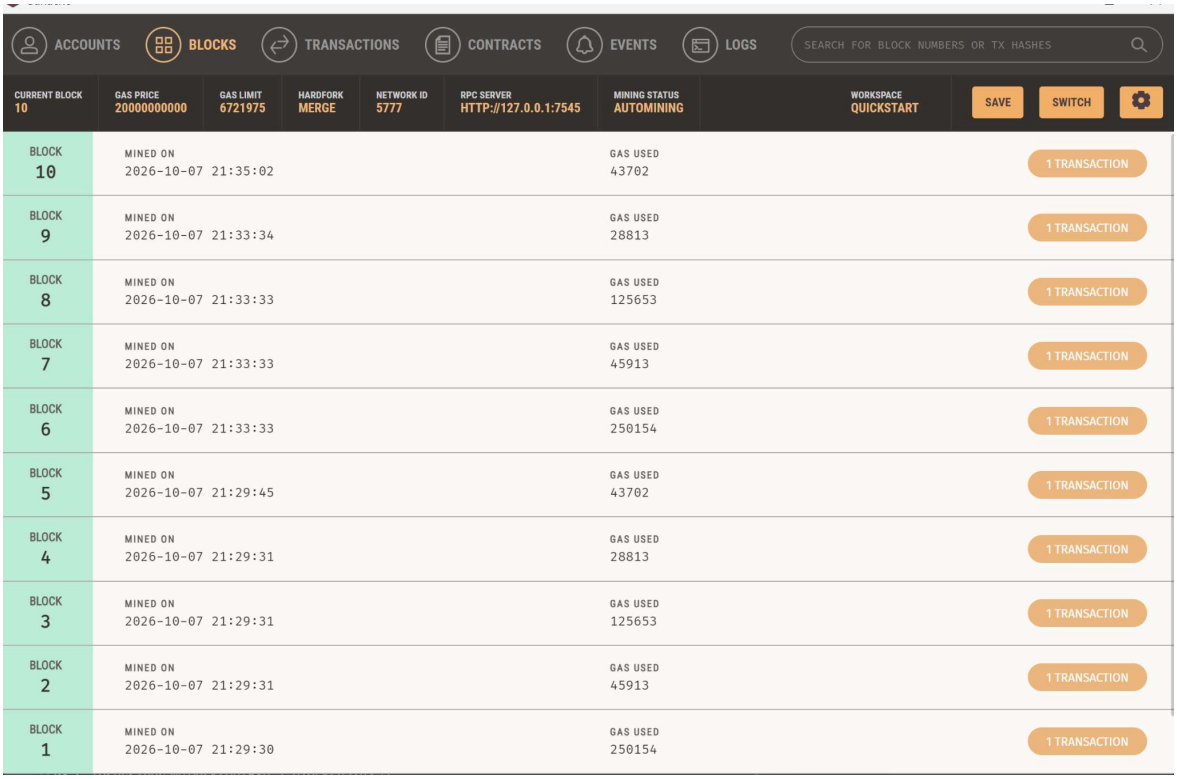
Step 6 — Verify the transactions in Ganache

The Transactions tab lists the contract creations and the contract calls with their transaction hashes, gas used and addresses.



Step 7 — Verify the blocks in Ganache

Each transaction was mined into its own block through automine, with the latest block being block 10.



Conclusion:

A local Ethereum development pipeline was successfully established using the **Truffle Suite** and **Ganache**. The SimpleStorage contract was compiled into bytecode and ABI artifacts and migrated to the private blockchain at 127.0.0.1:7545. A state-changing **transaction** (set) and a read-only **call** (get) were verified, confirming state persistence and block creation without real gas fees. The experiment gave a foundation in **local node setup, migration scripts, and DApp interaction workflows**.